The spurious wakeups mean that a thread calling `wait()` may wake even though no thread has called `notify_one()` or `notify_all()`. It's therefore recommended that the overload of `wait()` that takes a predicate is used in preference where possible. Otherwise, it's recommended that `wait()` be called in a loop that tests the predicate associated with the condition variable.

Synchronization

Calls to `notify_one(), notify_all(), wait(), wait_for()`, and `wait_until()` on a single `std::condition_variable_any` instance are serialized. A call to `notify_one()` or `notify_all()` will only wake threads that started waiting *prior* to that call.

Std::Condition_Variable_Any::Wait Member Function Overload That T- Takes a Predicate

Waits until the `std::condition_variable_any` is woken by a call to `notify_one()` or `notify_all()` and the predicate is `true`.

Declaration

```
template<typename Lockable,typename Predicate>
void wait(Lockable& lock,Predicate pred);
```

Preconditions

The expression `pred()` shall be valid and shall return a value that's convertible to `bool`. `Lockable` meets the `Lockable` requirements, and `lock` owns a lock.

Effects

As-if